LIN7076 – Foundations of Computational Linguistics

Logistic Regression

Adèle Hénot-Mortier

19/10/2026

Queen Mary University of London

Can you spot the phonological difference?

Obama's Congressional Black Caucus speech, September 2011:
https://www.youtube.com/watch?v=eMob_7C7i6o

Obama on what he hopes readers take away from his book, 2020:
https://youtu.be/F20ll0BDtTk?si=L_DzgNO446E5ne8n&t=79

Modeling “g-dropping’

- We’d like to predict if a word ending in *-ing* will be pronounced /ɪn/ (**no “g”**) or /ɪŋ/ (**“yes g”**).[1]
- Sensible factors share similarities with the ones we discussed last week:
  - speech `rate`;
  - `gender`;
  - `context` (incl. audience);
  - `age`;
  - socio-economic `status`.
- What would be the dependent/independent variables?

[1] Comparable phenomena: *wanna*-contraction; flapping (writer/rider), r-dropping (car/cah), h-dropping (has/’as), the PRICE vowel (price/proice/pras)...Can you think of other patterns, in English or another language you speak?

Predicting “g-dropping”

- We want to predict whether or not the “g” will be pronounced at the end of a word in *-ing*. It’s a **binary variable**, and we are dealing with **binary classification**. Can we use Naive Bayes? Can we use linear regression?
- We could use Naive Bayes, but cannot use linear regression, because such models predict a continuous (unbounded) variable, not 0 or 1!
- We’ll introduce the logistic regression model today, which is basically a linear regression **plus an extra transformation at the end, which will allow us to infer a binary class instead of a continuous variable**.

Plan for today

- Introducing a non-linearity: the sigmoid function
- Using the sigmoid for classification
- Fitting a logistic regression model with gradient descent

Logistic=linear+sigmoid

Logistic regression at a glance

A **logistic** regression predicts a binary dependent variable from the independent variable(s), assuming the relation between the two is linear, followed by one non-linearity.

- In fact logistic regression can be **generalized to predict nominal variables**, i.e. more than 2 classes.

Starting from a linear regression model

A logistic regression predicts a binary dependent variable from the independent variable(s), **assuming the relation between the two is linear**, followed by one non-linearity.

- Here is what a **linear regression** would look like, given our independent variables.

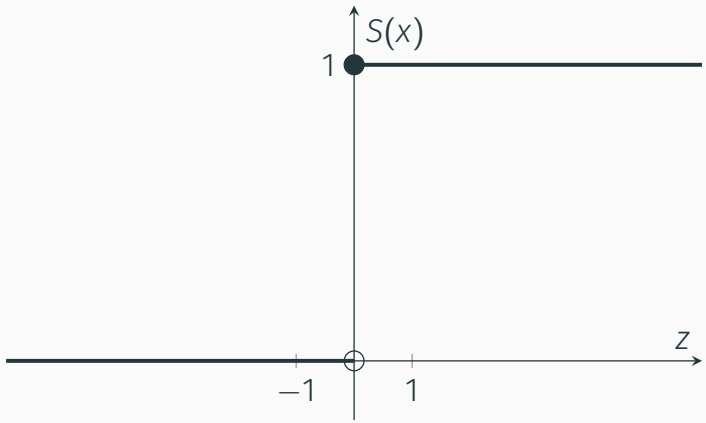
$$
\begin{aligned}
z &= \beta_1 x_1 + \ldots + \beta_n x_n + \alpha \\
z &= \beta_1 \texttt{rate} + \beta_2 \texttt{is_male} + \beta_3 \texttt{is_female} \\
&\quad + \beta_4 \texttt{context} + \beta_5 \texttt{age} + \beta_6 \texttt{status} + \alpha
\end{aligned}
$$

- If we don't do anything else, our prediction z will live in $]-\infty; +\infty[$, because the βs and α live in $]-\infty; +\infty[$.
- But we want our prediction to be 0 or 1! Idea: **we apply an extra transformation to z** that "squishes" it to live in $]0; 1[$.

The most intuitive squishing function

A logistic regression predicts a binary dependent variable from the independent variable(s), assuming the relation between the two is linear, **followed by one non-linearity**.

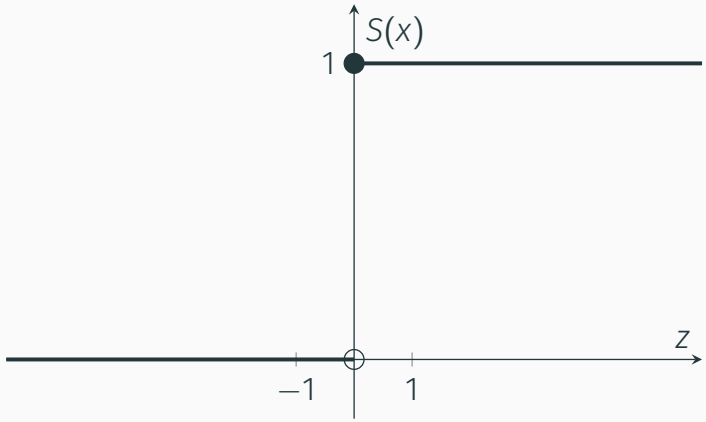
- We could assume z gets squished using a **step function**, mapping negative values of z to 0 and positive values to 1...



Thanks to ChatGPT

$$S(z) = \begin{cases} 0 \text{ if } z < 0 \\ 1 \text{ otherwise} \end{cases}$$

Issue with the “step” squishing



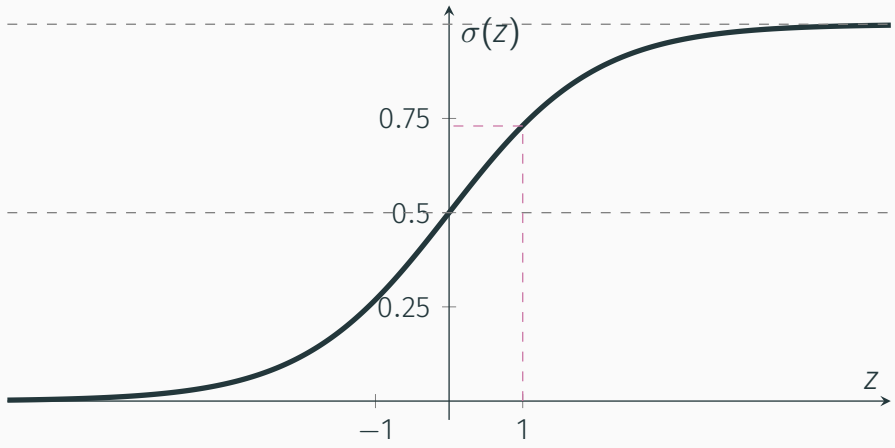
Thanks to ChatGPT

- The “step” at 0 is vertical: we “jump” from 0 to 1. This means **the slope is undefined** at $x = 0$.
- And anywhere else the **slope is** 0 (flat function).
- But we’ll see that the algo used to fit logistic regression models (gradient descent) needs functions with **finite, non-zero slopes…** So a step like this is no good.

- Also, mapping all negative (resp. positive) values to a single number (0/1) **erases interesting intermediate information**–how *probable* the predicted class is.

A smoother, smarter squishing

- We want a squishing function that **looks like a step function, but is never always flat and has no sudden "jump".**
- The **sigmoid** function ($\sigma(z) = \frac{1}{1+e^{-z}}$) achieves this. It's equal to .5 for $z = 0$, converges to 0 for $z \to -\infty$ and to 1 for $z \to +\infty$.



Thanks to ChatGPT

- $\sigma(z)$ essentially tells you **how probable class 1 is, for any z.** It's symmetric around point (0, .5), which means $\sigma(z) = 1 - \sigma(-z)$.[2]

[2] From an optimization perspective, it's also very nice because its slope has a nice formula equal to $\sigma(z) \times (1 - \sigma(z))$, for any z.

Practice 1

$$z = \beta_1 \texttt{rate} + \beta_2 \texttt{is_male} + \beta_3 \texttt{is_female} + \beta_4 \texttt{context} + \beta_5 \texttt{age} + \beta_6 \texttt{status} + \alpha$$

- Suppose our speaker is a 40 y.o., non-binary, upper middle-class (assumed level 2) speaker, talking in an informal (assumed class 0) context at 120wpm. For concreteness, we do not standardize these variables.
- We have $\beta_1 = -.1$, $\beta_5 = -.2$, $\beta_6 = 5$, $\alpha = 10$. What are z and $\sigma(z)$?

$$\begin{aligned} z &= -.1 \times 120 + 0 + 0 + 0 + -.2 \times 40 + 5 \times 2 + 10 \\ &= -12 - 8 + 10 + 10 = 0 \\ \sigma(z) &= .5 \end{aligned}$$

- Probability to drop “g” is 50/50!

Practice 2

$$z = \beta_1 \texttt{rate} + \beta_2 \texttt{is_male} + \beta_3 \texttt{is_female} + \beta_4 \texttt{context} + \beta_5 \texttt{age} + \beta_6 \texttt{status} + \alpha$$

- Suppose our speaker is a 60 y.o., female, upper class (assumed level 3) speaker, talking in a formal (assumed class 1) context at 100wpm. For concreteness, we do not standardize these variables.
- We have $\beta_1 = -.1$, $\beta_3 = 5$, $\beta_4 = 20$, $\beta_5 = -.2$, $\beta_6 = 5$, $\alpha = 10$. What are z and $\sigma(z)$?

$$\begin{aligned} z &= -.1 \times 100 + 0 + 5 \times 1 + 20 \times 1 - .2 \times 60 + 5 \times 3 + 10 \\ &= -10 + 5 + 20 - 12 + 15 + 10 = 38 \\ \sigma(z) &\simeq 1 \end{aligned}$$

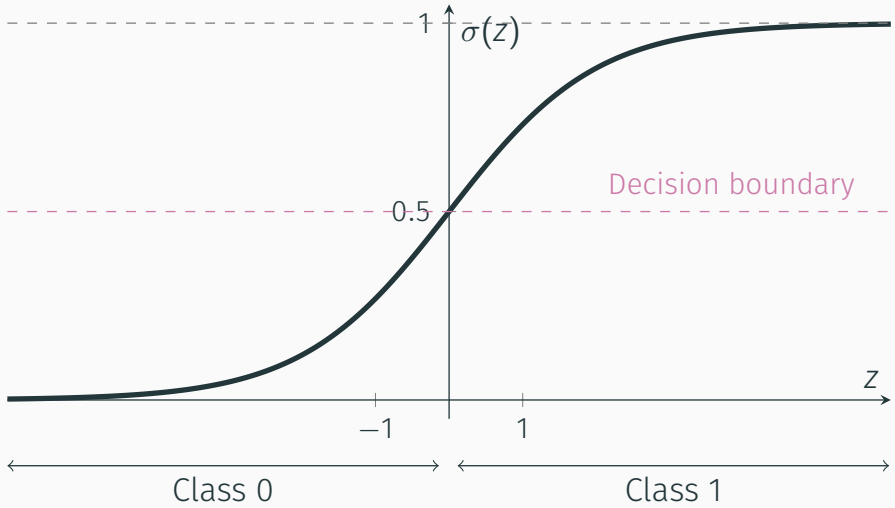
- Probability to keep “g” is virtually 1!

Logistic classification

The sigmoid's decision boundary

- The sigmoid function returns the **probability to predict class 1** (for us, predict "yes g").
- To get a clear-cut class, one can round this probability to the nearest integer !

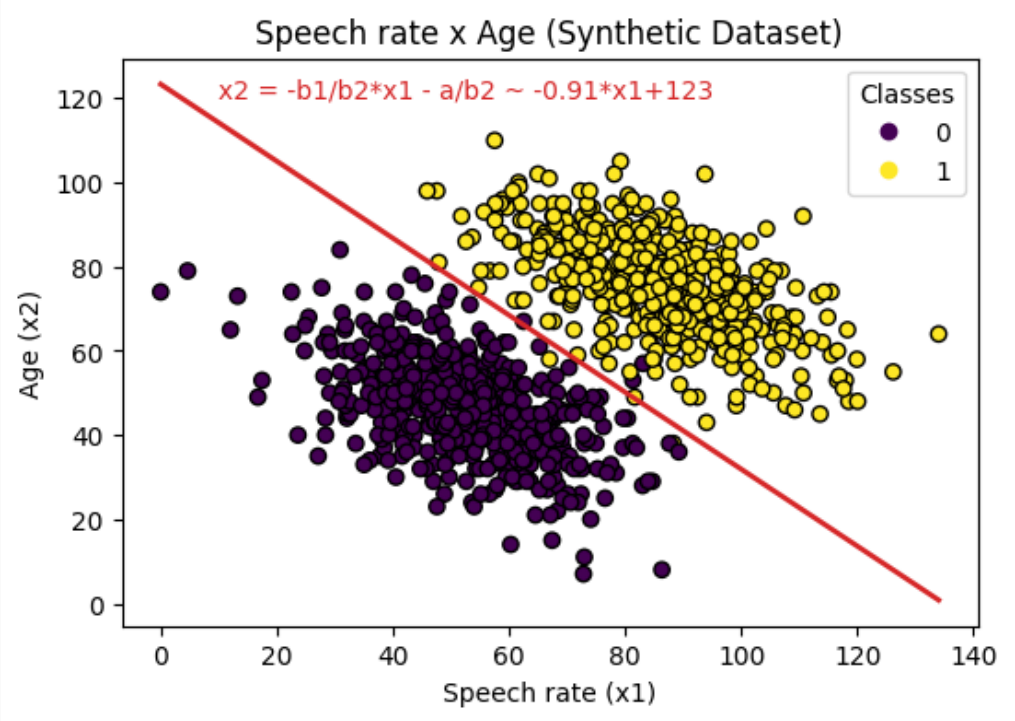
$$\hat{c} = \begin{cases} 1 \text{ if } \sigma(z) > .5 \\ 0 \text{ otherwise} \end{cases}$$



Thanks to ChatGPT

Decision boundary from a scatter plot

- Assuming 2 independent variables x_1 and x_2, **logistic** regression is all about fitting a line **maximally separating the two classes in a scatter plot linking x_1 and x_2.**



- If $z = \beta_1 \times x_1 + \beta_2 \times x_2 + \alpha$, then the separator will have equation $x_2 = -\frac{\beta_1}{\beta_2} x_1 - \frac{\alpha}{\beta_2}$

Deriving the regression's parameters from its separator

- Assume the following blue separator. Let's infer the model's parameters from it.

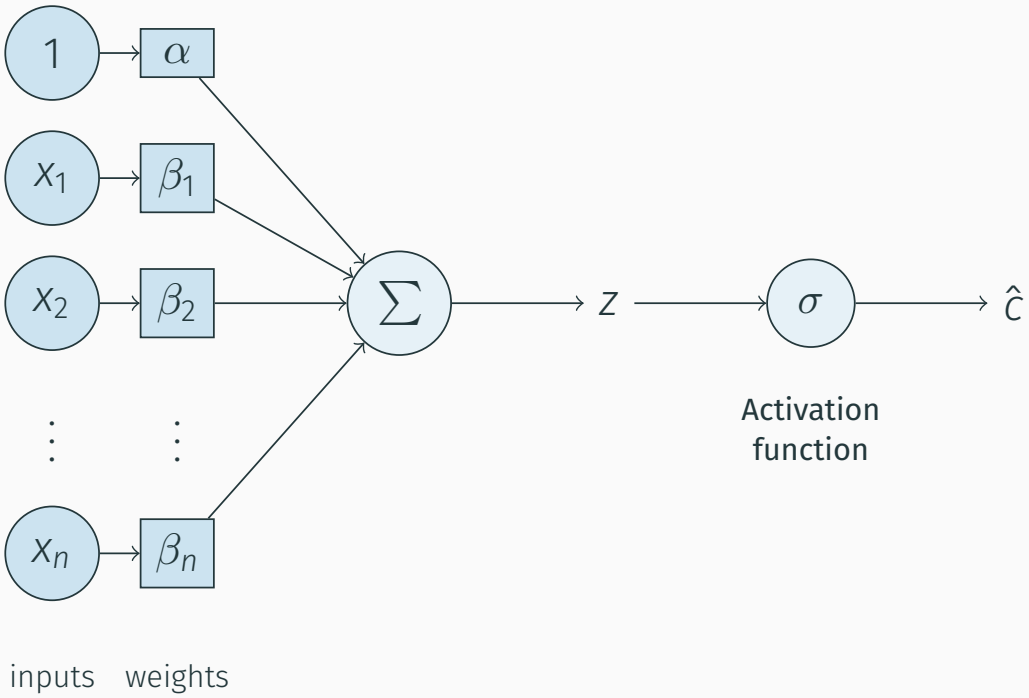


- Curve crosses y-axis at $y = +3$, so intercept is $+3$;
- Curve goes -2 on y for each $+1$ on x: slope is -2.
- So the separator has equation $y = -2x + 3$.

- We know that the slope is $\frac{-\beta_1}{\beta_2}$ and the intercept is $\frac{-\alpha}{\beta_2}$, with β_1, β_2 and α the parameters of the underlying logistic regression.
- So $\frac{-\beta_1}{\beta_2} = -2$ and $\frac{-\alpha}{\beta_2} = 3$. Any β_1, β_2 and α verifying this would yield this separator.
- For instance, setting β_2 to 1 arbitrarily, we would then have $\beta_1 = 2$ and $\alpha = -3$. So $(\beta_1, \beta_2, \alpha) = (2, 1, -3)$ works.

Link with neural nets

- Logistic regression is **at the core of basic neural networks**.
- Roughly, basic "feed-forward" neural nets are stacked parallelized logistic regressions, **except the non-linearity is not necessarily sigmoid**.



Logistic regression is a one-layer, one-unit neural net, with a sigmoid **activation function**.

Fitting a logistic regression

A supervised training algorithm

- Logistic regression is **supervised**: we learn the parameters of the model (βs, α) based on known input-output pairs (datapoints) of the form $((x_1, x_2, ..., x_n), c)$, with x_i the independent variables, and c the ground-truth class.
- Datapoint example: a 60 y.o., female, upper class speaker, talking in a formal context at 100wpm (=independent variables x_i) and who drops her "g" (dependent variable $c = 0$).
- To learn from these datapoints, we'll need two core ingredients:
  - A measure (**loss**) penalizing wrong predictions.
  - A way to **propagate** penalties to the parameters of the model (βs and α) in order to gradually improve the predictions.

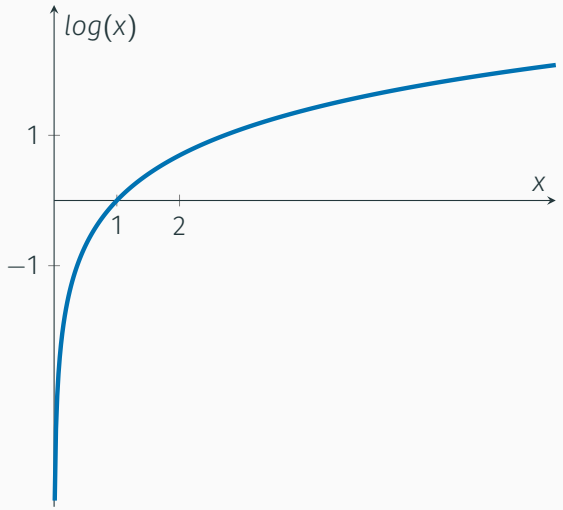
Measuring performance: core intuition

- To improve the model, we need a measure **of how well we are doing** with our predictions. Intuitively, for any given training datapoint:
  - If the ground-truth class c is 0 and we predict $\hat{c} = 0$, we're good;
  - If the ground-truth class c is 1 and we predict $\hat{c} = 1$, we're good;
  - If the ground-truth class c and the predicted class $\hat{c}$ differ, we're not good.
- We need a measure that **aggregates** these intuitions into one single formula, and is **more fine-grained** than just "we're good" vs. "we're not good".

Measuring performance: by-class desiderata

- Our ground-truth classes c are **binary**, i.e. are 0 or 1.
- Recall $\sigma(z)$ corresponds to the **probability that the input is associated with class** 1. This probability belongs to $]0;1[$. If $\sigma(z) > .5$ we predict $\hat{c} = 1$, and $\sigma(z) \leq .5$ we predict $\hat{c} = 0$.
- For any datapoint, $\sigma(z)$ **and** c **must be as close as possible**:
  - if $c = 0$ we want $\sigma(z) \simeq 0$, i.e. $1 - \sigma(z) \simeq 1$;
  - and if $c = 1$ we want $\sigma(z) \simeq 1$.

- For theoretical and analytical reasons, we "log" these equalities (log is increasing and s.t. $log(1) = 0$ and $log(0) = -\infty$):
  - if $c = 0$ we want $\boldsymbol{log}(1 - \sigma(z)) \simeq \mathbf{0}$;
  - and if $c = 1$ we want $\boldsymbol{log}(\sigma(z)) \simeq \mathbf{0}$.



Measuring performance: cross-entropy loss

- if $c = 0$ we want $\boldsymbol{log}(1 - \sigma(z)) \simeq \mathbf{0}$ and if $c = 1$ we want $\boldsymbol{log}(\sigma(z)) \simeq \mathbf{0}$.
- We can merge these two cases into one single formula to get the **cross-entropy (CE) loss**:

$$L_{\text{CE}} = -(\underbrace{c \times log(\sigma(z))}_{\neq 0 \text{ iff } c=1} + \underbrace{(1-c) \times log(1-\sigma(z))}_{\neq 0 \text{ iff } c=0})$$

- The next Slide double-checks this loss truly penalizes wrong predictions (and spares right predictions), i.e. is **high** when predicted and ground-truth classes **mismatch**, and $\simeq 0$ otherwise.

Cross-entropy loss sanity check

$$L_{\text{CE}} = -(\underbrace{c \times log(\sigma(z))}_{\neq 0 \text{ iff } c=1} + \underbrace{(1-c) \times log(1-\sigma(z))}_{\neq 0 \text{ iff } c=0})$$

- If $c = 0$, $L_{\text{CE}} = -(0 + (1-0) \times log(1-\sigma(z))) = -log(1-\sigma(z))$
  - If $\sigma(z) \simeq 0$, $1-\sigma(z) \simeq 1$ and $-log(1-\sigma(z)) \simeq 0$. Our loss is **low**, consistent with the fact our prediction and ground-truth **match**.
  - If $\sigma(z) \simeq 1$, $1-\sigma(z) \simeq 0$ and $-log(1-\sigma(z)) \simeq +\infty$. Our loss is **high**, consistent with the fact our prediction and ground-truth **mismatch**.
- If $c = 1$, $L_{\text{CE}} = -(1 \times log(\sigma(z)) + 0) = -log(\sigma(z))$
  - If $\sigma(z) \simeq 0$, $-log(\sigma(z)) \simeq +\infty$. Our loss is **high**, consistent with the fact our prediction and ground-truth **mismatch**.
  - If $\sigma(z) \simeq 1$, $-log(\sigma(z)) \simeq 0$. Our loss is **low**, consistent with the fact our prediction and ground-truth **match**.
- In brief, L_{CE} is guaranteed to be $\simeq 0$ if our prediction is right, and high if our prediction is wrong–no matter the class!

Unpacking cross-entropy loss

- The cross-entropy loss function tells us how well we're predicting. Now we need to use this information to **update our model parameters** (βs and α) **in a direction that improves our next predictions**!
- Let's unpack L_{CE} to reveal how it depends on these parameters.

$$
\begin{aligned}
z &= \beta_1 x_1 + ... + \beta_n x_n + \alpha \\
L_{CE} &= -(c \times log(\sigma(z)) + (1 - c) \times log(1 - \sigma(z))) \\
&= (c \times log(\sigma(\beta_1 x_1 + ... + \beta_n x_n + \alpha)) \\
&\quad +(1 - c) \times log(1 - \sigma(\beta_1 x_1 + ... + \beta_n x_n + \alpha)))
\end{aligned}
$$

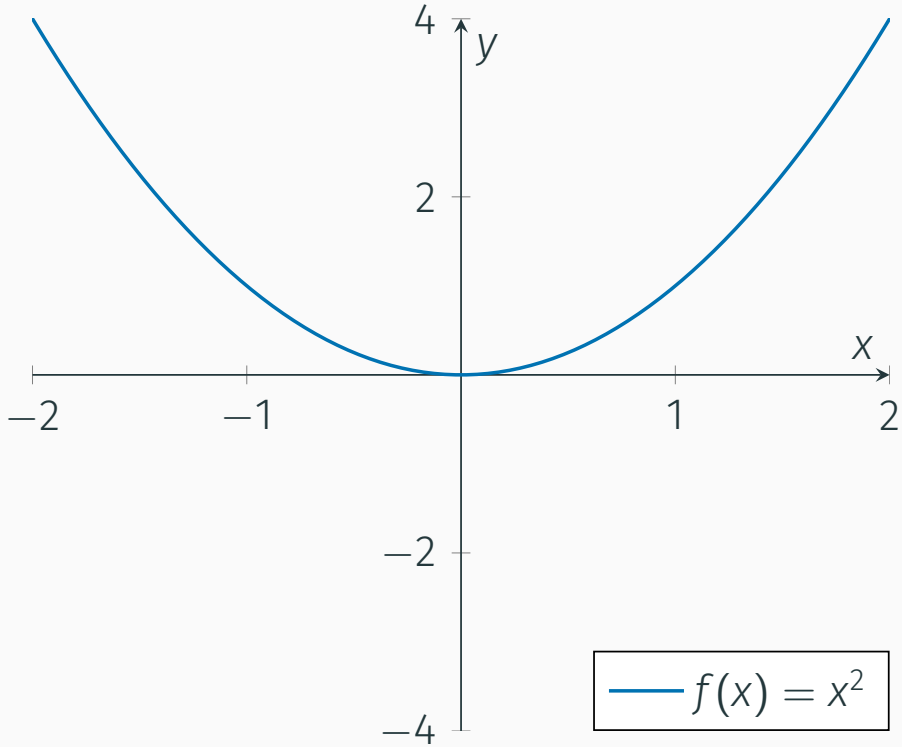
- For any fixed datapoint, i.e. fixed values of x_1, ..., x_n, L_{CE} is a function of the βs and α: $L_{CE} = L_{CE}(\beta_1, ..., \beta_n, \alpha)$.
- This means that varying a β, or α, leads to a variation of L_{CE}.

Gradient descent intuition

- For any fixed datapoint, $L_{\text{CE}}(\beta_1, ..., \beta_n, \alpha)$ varies with the βs and α.
- Ideally we want $L_{\text{CE}}(\beta_1, ..., \beta_n, \alpha)$ to be $\simeq 0$, because this means perfectly accurate predictions.
- Idea: for each datapoint, slightly update the βs and α **in a direction that decreases L_{CE}**.
- Hopefully after going through many datapoints, the βs and α converge to values that make L_{CE} closer to 0 and improve our classification performance.
- Let's now see what it means to "update in the decreasing direction" based on a simple example.

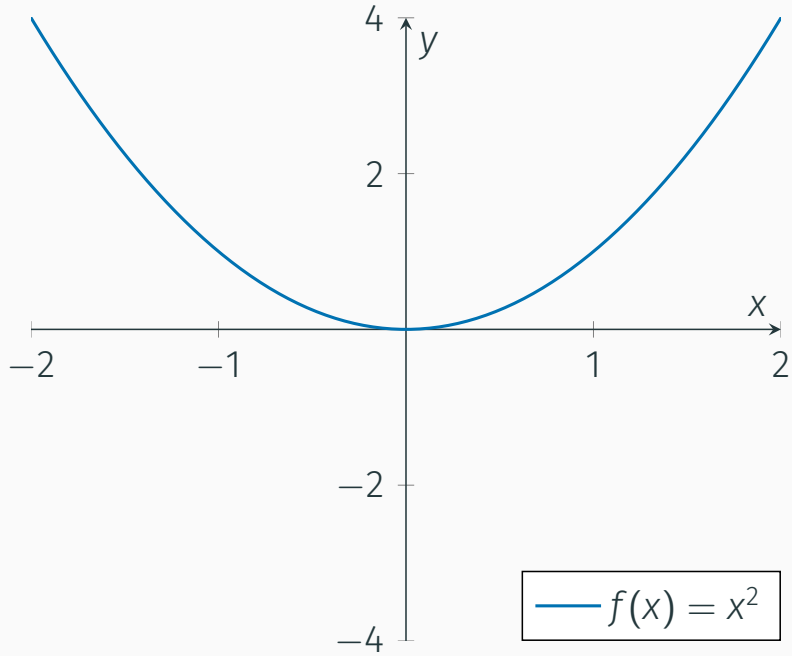
Gradient of univariate functions

- Let's focus on a function with only one variable, $f(x) = x^2$. This function is a parabole: it decreases for negative values of x, hits its minimum 0 at $x = 0$, then symmetrically increases for positive values of x.



Gradient of a single-variable function

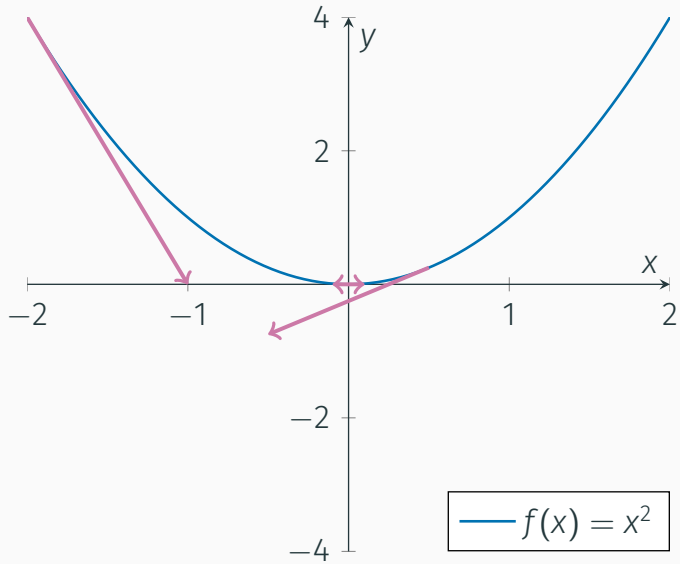
- Suppose we leave a ball with no speed on this curve at $x = -2$ what would it do? And how fast?
- What if we put a ball on this curve at $x = 0$? $x = .5$?



- The ball would bo down the curve with a speed that would depend on how steep the slope is! If the ball is left at a minimum (here $x = 0$), it stays in place, i.e. has no speed.

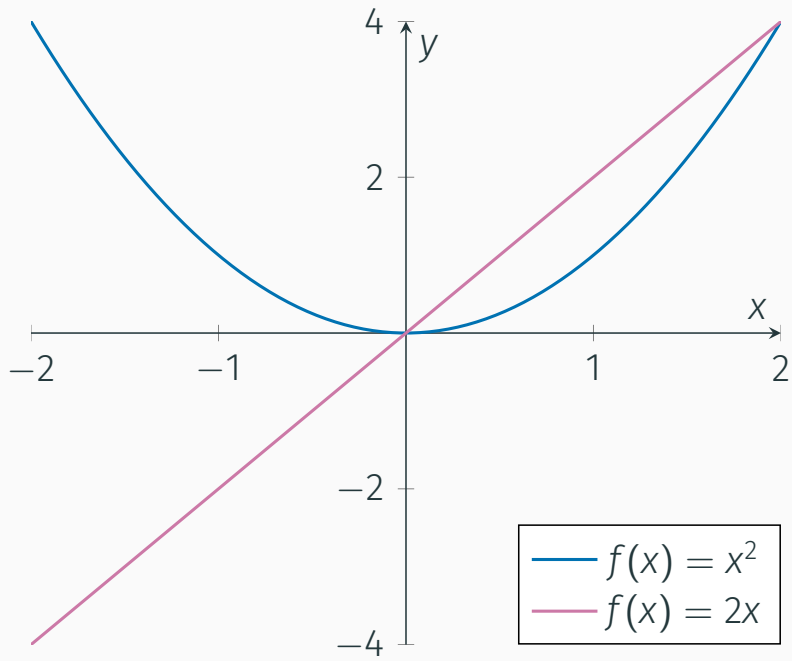
Gradient of a single-variable function

- The **gradient** is a generalization of the concept of **slope** used in linear functions.
- It corresponds to the **slope of the tangent to the curve**, at any given point. It's positive (resp. negative) for locally increasing (resp. decreasing) functions.
- The **negative gradient** of a function (red arrows on the plot) quantifies what a ball would do, when left at any point of the curve. It points to the decreasing direction, and its magnitude indicates the steepness of the local slope.



Gradient of a single-variable function

- The gradient of many functions can be computed by composing known formulas. For $f(x) = x^2$, the gradient is $2x$, i.e. it's -4 (=steeply decreasing) for $x = -2$, 0 (=minimum/maximum) for $x = 0$, and 1 (=slightly increasing) for $x = .5$...



Gradient descent

- The negative gradient of a function **points to the direction of decrease of the function**.
- Let's consider one parameter of our logistic regression, e.g. β_i.
- We can compute the negative gradient of L_{CE}, seen as a function of β_i alone: it's equal to $(c - \sigma(z))x_i$, i.e. it's **proportional to how close our prediction is to the ground-truth** $(c - \sigma(z))$, **modulated by the value of the feature itself** (x_i).
- The value of this gradient then tells us in which direction, and by how much, we should update β_i in order to decrease L_{CE}!
- Each time a datapoint is processed, each β_i is "moved" in the direction of the negative gradient by a certain factor η: $\beta_i := \beta_i + \eta(c - \sigma(z))x_i$.
- α as well: $\alpha := \alpha + \eta(c - \sigma(z))$.

Gradient descent practice 1

$$\beta_i := \beta_i + \eta(c - \sigma(z))x_i \qquad \alpha := \alpha + \eta(c - \sigma(z))$$

- Suppose we are just starting to fit our model and our prediction is crap, e.g. our ground-truth is $c = 1$, but we predict $\sigma(z) = .2$. Suppose x_i is the feature `context` and is equal to 1. $\eta = .1$.
- Will β_i increase or decrease after the update? How would using this new β_i affect our current prediction?
- $\eta(c - \sigma(z))x_i = .1 \times (1 - .2) \times 1 = .1 \times .8 = .08$. It's an increase. All other things being equal, increasing β_i would increase z, and increase $\sigma(z)$, which means our prediction would get closer to the ground-truth (1).

Gradient descent practice 2

$$\beta_i := \beta_i + \eta(c - \sigma(z))x_i \qquad \alpha := \alpha + \eta(c - \sigma(z))$$

- Suppose have been running gradient descent for a while on several datapoints and our predictions got better, and now our ground-truth is $c = 0$, and we predict $\sigma(z) = .3$. Suppose that again x_i is the feature `context` and is equal to 1. $\eta = .1$.
- Will β_i increase or decrease? How would using this new β_i it affect our prediction? How big is the increase/decrease when compared to the one computed on the previous slide?
- $\eta(c - \sigma(z))x_i = .1 \times (0 - .32) \times 1 = .1 \times (-.3) = -.03$. It's a decrease. All other things being equal, decreasing β_i would decrease z, and decrease $\sigma(z)$, which means our prediction would get closer to the ground-truth (0). The magnitude of the decrease is smaller than the one computed before, essentially because we were already quite close to the ground truth!

Lab

https://colab.research.google.com/drive/13JDPI-HQQsBM1BSxm7YUPCd_YB58VbuJP?usp=sharing

Extras

Sigmoid slope/derivative calculation

- We are looking for the derivative ($'$) of our sigmoid function:

$$\sigma'(z) = \left(\frac{1}{1+e^{-z}}\right)' = \left(\frac{1}{u(z)}\right)' \text{ with } u(z) = 1+e^{-z}$$

- We know that the derivative of $\frac{1}{u}$ is $\frac{-u'}{u^2}$.

$$\sigma'(z) = \frac{-u'(z)}{u(z)^2} \text{ with } u(z) = 1+e^{-z}$$

- Now we need to compute the derivative of $u(z) = 1+e^{-z}$.

$$u'(z) = (1+e^{-z})' = (e^{-z})' = (e^v)' \text{ with } v(z) = -z$$

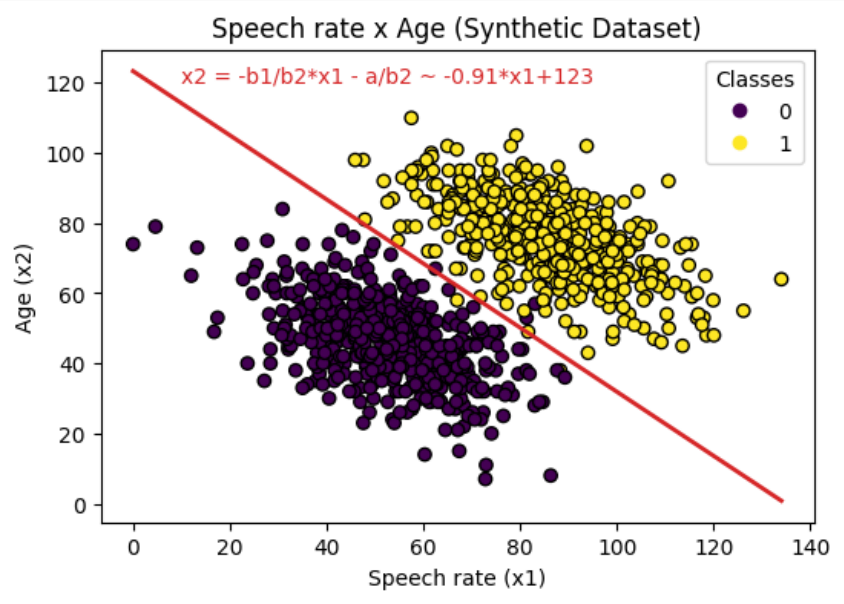
- We know that the derivative of e^v is $v'e^v$ and $v'(z) = (-z)' = -1$. S now we can fully express $u'(z)$:

$$u'(z) = v'(z)e^{v(z)} = -e^{-z}$$

- Rewriting this into our expression of $\sigma'(z)$...

$$\sigma'(z) = \frac{-u'(z)}{u(z)^2} = \frac{-(-e^{-z})}{(1+e^{-z})^2} = \sigma(z)\frac{e^{-z}}{1+e^{-z}} = \sigma(z)\frac{e^{-z}+1-1}{1+e^{-z}} = \sigma(z)(1-\sigma(z))$$

Deriving the separator's equation in 2D



- The decision boundary is the set of points (x_1, x_2) s.t. $\sigma(z(x_1, x_2)) = .5$, i.e. we are undecided between class 0 and class 1 for these points. Let's unpack this.

$$
\begin{aligned}
\sigma(z(x_1, x_2)) = .5 \quad &\iff \quad z(x_1, x_2) = 0 \\
&\iff \quad \beta_1 x_1 + \beta_2 x_2 + \alpha = 0 \\
&\iff \quad \beta_2 x_2 = -\beta_1 x_1 - \alpha \\
&\iff \quad x_2 = \underbrace{-\frac{\beta_1}{\beta_2}}_{\textit{slope}} x_1 \quad \underbrace{-\frac{\alpha}{\beta_2}}_{\textit{intercept}}
\end{aligned}
$$